

Propuesta de Refactorización: Razonamiento y Memoria del Agente KAI

Proyecto: KognitoAI / KAI OS **Versión:** 1.0 **Fecha:** 20 de julio de 2026 **Autor:** Arquitectura KAI **Estado:** Propuesta para revisión

1. Resumen Ejecutivo

KAI posee una arquitectura de memoria y razonamiento híbrida (vectorial + grafo) que ya se alinea con el patrón ganador del estado del arte 2025–2026. Sin embargo, la investigación reciente identifica brechas críticas en **provenance/trust, bi-temporalidad, memoria episódica, active forgetting, conflict resolution y evaluación estructurada.**

Esta propuesta describe 6 líneas de refactorización **aditivas** (sin ruptura de schema) que hardened la arquitectura actual, mitigan el riesgo ASI06 (Memory Poisoning) y posicionan a KAI a la vanguardia en seguridad y precisión cognitiva.

Objetivo: Elevar la confiabilidad, trazabilidad y capacidad de aprendizaje autónomo del agente sin reemplazar la base existente.

2. Diagnóstico de la Arquitectura Actual

2.1 Componentes Identificados

Componente	Tecnología	Función Actual
Memoria Working	PostgreSQL + LangGraph State	Hilo inmediato, ventana de tokens
Memoria Semántica	Qdrant (text-embedding-004)	Recuperación difusa por similitud coseno
Memoria Relacional	Neo4j	Entidades, decisiones, travesías multi-hop
Extracción de Conocimiento	SLM embebido Qwen2.5-3B (GPU CUDA)	Filtrado de ruido y escritura en Neo4j
Orquestación	LangGraph	Grafo dirigido de estados (unified → graph → model)

2.2 Brechas Críticas Detectadas

1. **Sin provenance ni trust scoring:** Los nodos Neo4j registran `confidence` y `source_document`, pero no hay trazabilidad de *quién* extrajo el nodo, *con qué modelo*, ni un score agregado de confianza.
2. **Sin bi-temporalidad:** Solo `created_at/updated_at`. No soporta hechos que dejan de ser ciertos (ej. versiones, estados cambiantes).
3. **Sin memoria episódica:** No hay recuperación por recencia/episodio independiente del grafo semántico.
4. **Sin active forgetting:** El SLM extractor solo ejecuta `ADD`. No tiene acciones `DELETE/REFINE` para reconciliar contradicciones.
5. **Sin conflict resolution:** `_combine_contexts` concatena sin resolver choques entre fuentes.
6. **Sin benchmark de memoria:** No se evalúa LoCoMo/LongMemEval/BEAM; las métricas actuales no capturan memoria temporal.

3. Propuestas de Refactorización

3.1 Provenance & Trust Tracking en Neo4j

Objetivo: Mitigar ASI06 (Memory Poisoning) trazando origen y confiabilidad de cada nodo/relación.

Cambios en schema:

```
// Migración: añadir a todos los nodos Entity/DocumentChunk
ALTER NODE Entity ADD provenance_source TEXT;
ALTER NODE Entity ADD provenance_method TEXT;    // 'slm_extractor' |
'llm_fallback' | 'user'
ALTER NODE Entity ADD trust_score FLOAT DEFAULT 0.8;
ALTER NODE Entity ADD extraction_model TEXT;      // 'Qwen2.5-3B-Instruct'
```

Cambios en código:

- `neo4j_adapter.py:_add_entities_to_neo4j` → incluir `provenance_*` y `trust_score` en props.
- Nueva función `_compute_trust(confidence, source, method)` que descuenta nodos de `slm_extractor` con `confidence < 0.6` y aplica bonus a `source_document` verificado.
- `graph_reasoning_node.py` → filtrar travesías Cypher por `trust_score > $min_trust` (configurable, default 0.5).

Beneficio: Trust scoring como gate de recuperación; provenance para auditoría.

3.2 Capa Bi-Temporal en Relaciones Clave

Objetivo: Soportar hechos que dejan de ser ciertos sin perder historial.

Cambios en schema:

```
// Extender relaciones con validez temporal
ALTER RELATIONSHIP REL ADD valid_from TIMESTAMP;
ALTER RELATIONSHIP REL ADD valid_to TIMESTAMP;
ALTER RELATIONSHIP REL ADD is_current BOOLEAN DEFAULT TRUE;
```

Cambios en código:

- `neo4j_adapter.py:_add_relationships_to_neo4j` → setear `valid_from = created_at, valid_to = None`.

- Operación de actualización (cuando SLM detecta contradicción):

```
MATCH (s)-[r:REL]->(t) WHERE id(r)=$rid
SET r.valid_to = datetime(), r.is_current = False
CREATE (s)-[:REL {valid_from: datetime(), is_current:True, ...}]->(t)
```

- `graph_reasoning_node.py` → filtrar expansión por `is_current = True`.

Beneficio: Razonamiento temporal preciso; soporta versionado de conocimiento.

3.3 Memoria Episódica Ligera (Postgres + Embedding Interno)

Objetivo: Recuperación por recencia/episodio independiente del grafo semántico.

Cambios en schema:

```
CREATE TABLE episodic_memory (
    id UUID PRIMARY KEY,
    account_id UUID,
    workspace_id UUID,
    event_text TEXT,
    embedding vector(384),
    occurred_at TIMESTAMPTZ,
    episode_type TEXT -- 'chat' | 'task' | 'decision'
);
```

Cambios en código:

- Reutilizar `core/embedding_manager.py` (`KognitoInternalEmbeddingService`, `SentenceTransformer`) para generar embeddings.
- `enhanced_memory_manager.py:_get_traditional_context` → añadir branch episódico que consulta `episodic_memory` y blend con contexto semántico (peso recencia 0.3).
- `memory_graph_processor.py` → poblar `episodic_memory` desde memorias no procesadas.

Beneficio: Recuperación temporal-aware; complementa grafo con "memoria de experiencias".

3.4 Active Forgetting en el SLM Extractor

Objetivo: Permitir al agente borrar o refinar conocimiento obsoleto/contradictorio.

Cambios en prompt del SLM:

```
Responde JSON con acciones:  
{"action":"ADD", ...} | {"action":"DELETE", "target_id":"...",  
"reason":"..."} |  
{"action":"UPDATE", "target_id":"...", "new_props":{...}}
```

Cambios en código:

- `embedded_slm_extractor.py` → nuevo método `reconcile(new_msg, existing_nodes)` que detecta contradicciones y devuelve acciones DELETE/UPDATE.
- `knowledge_extraction_node.py` → hook para aplicar DELETE vía `neo4j_adapter` (nueva función `_delete_entity`).
- `neo4j_adapter.py` → implementar `_delete_entity(target_id)` y `_update_entity(target_id, new_props)`.

Beneficio: El agente puede "olvidar" activamente; reduce acumulación de ruido y contradicciones.

3.5 Conflict Resolution en Combinación de Contextos

Objetivo: Resolver hechos contradictorios entre grafo y contexto tradicional antes de enviar al LLM.

Cambios en código:

- `enhanced_memory_manager.py:_combine_contexts` → añadir detección de conflictos:

```
conflicts = self._detect_conflicts(graph_ctx, traditional_ctx)  
if conflicts:  
    resolution = await llm.arbitrate(conflicts,  
prefer=high_trust_score)  
    ctx["resolved_conflicts"] = resolution
```

- `_detect_conflicts` compara entidades con mismo `name` pero props opuestos y diferentes `valid_from` (de 3.2).

Beneficio: Reduce alucinaciones por contexto contradictorio; mejora consistencia.

3.6 Benchmark Interno de Memoria

Objetivo: Medir objetivamente la calidad de memoria temporal y resistencia a poisoning.

Cambios en código:

- Extender

`skills/user_workspace_KognitoAI/kai_measurement_skill/scripts/measurement_engine.py`:

```
async def measure_memory_recall(self, dataset="LoCoMo-sample"):
    # Inyecta episodios en episodic_memory (3.3)
    # Consulta graph_reasoning_node con filtros trust (3.1)
    # Compara contra ground_truth temporal
    return {"temporal_acc": ..., "poison_resist": ...}
```

- Reutilizar `config.json` existente para añadir `memory_benchmarks`.

Beneficio: Métricas cuantificables; baseline para iteraciones futuras.

4. Plan de Implementación por Fases

Fase	Duración	Entregables	Dependencias
Fase 1: Trust & Provenance	2 semanas	Schema Cypher extendido, <code>_compute_trust</code> , filtro en <code>graph_reasoning_node</code> , tests unitarios	Ninguna
Fase 2: Bi-Temporalidad	2 semanas	Migración Cypher, operación UPDATE en <code>neo4j_adapter</code> , filtro <code>is_current</code>	Fase 1 (usa <code>trust_score</code> para preferir hechos vigentes)
Fase 3: Memoria Episódica	3 semanas	Tabla Postgres, embedding service, blend en <code>_get_traditional_context</code> , pipeline de poblamiento	Ninguna (paralelizable)
Fase 4: Active Forgetting	2 semanas	Prompt extendido SLM, <code>reconcile()</code> , <code>_delete_entity/_update_entity</code> en adapter	Fase 2 (usa bi-temporalidad para marcar obsolescencia)
Fase 5: Conflict Resolution	1 semana	<code>_detect_conflicts</code> , arbitraje LLM, integración en <code>_combine_contexts</code>	Fases 1 y 4
Fase 6: Benchmark	2 semanas	Suite LoCoMo/LongMemEval, reportes automatizados, dashboard	Fases 1-5

Esfuerzo total estimado: 12 semanas (1 desarrollador full-stack + 1 ingeniero ML para Fase 3/6).

5. Matriz de Priorización

Propuesta	Urgencia	Esfuerzo	Mitigación	Prioridad
3.1 Provenance & Trust	Alta	Bajo	ASI06 (Poisoning)	P0
3.2 Bi-Temporalidad	Alta	Medio	Conflictos, versionado	P1
3.6 Benchmark	Media	Medio	Medición, baseline	P2
3.3 Memoria Episódica	Media	Alto	Recencia, experiencias	P3
3.4 Active Forgetting	Media	Medio	Olvido activo, ruido	P4
3.5 Conflict Resolution	Baja	Bajo	Consistencia	P5

6. Riesgos y Mitigación

Riesgo	Probabilidad	Impacto	Mitigación
Migración Cypher rompe grafos existentes	Media	Alto	Script de migración idempotente con rollback; ejecutar en staging primero
SLM Qwen2.5-3B no genera acciones DELETE/UPDATE consistentes	Media	Medio	Fallback a LLM rápido para reconciliación; threshold de confianza
Overhead de embedding en Postgres degrada latencia	Baja	Medio	Índices HNSW en <code>episodic_memory.embedding</code> ; cache en <code>enhanced_memory_manager</code>
Trust scoring muy agresivo elimina nodos válidos	Baja	Alto	Calibración con dataset etiquetado; <code>min_trust</code> configurable por workspace

7. Conclusión y Próximos Pasos

La arquitectura de KAI es sólida en su base híbrida (vector + grafo + SLM local). Esta refactorización no la reemplaza, sino que **añade capas de confianza**,

temporalidad y aprendizaje autónomo que la industria ya considera estándar en 2026.

Próximos pasos recomendados:

7. Aprobar Fase 1 (Trust & Provenance) como sprint inmediato.
8. Ejecutar benchmark base (Fase 6) para cuantificar estado actual.
9. Revisar y ajustar `min_trust` y políticas de bi-temporalidad con equipo de producto.

Anexos:

- Anexo A: Scripts de migración Cypher detallados
- Anexo B: Prompt extendido para SLM extractor
- Anexo C: Esquema de tabla `episodic_memory` con índices
- Anexo D: Configuración de benchmarks LoCoMo/LongMemEval